



UNIVERSIDAD DE MURCIA

FACULTAD DE INFORMÁTICA

ECO MARKET

Sustainable Second-Hand Marketplace

Course 2025–2026

Authors:

Sara Macrì (1.2)

Danijel Merlo (1.1)

Subject:

Desarrollo de Aplicaciones Web

Professors:

Jose Antonio Miñarro Gimenez

Salvador Manzanares Guillén

22 May 2026

Contents

1	Introduction and Objectives	2
1.1	Technology Summary	2
1.2	General System Diagram	2
2	Work Distribution and Task Table	3
3	System Requirements	4
3.1	Functional Requirements	4
3.2	Non-Functional Requirements	5
4	Architecture and Technological Choices	5
4.1	Frontend	5
4.2	Backend	5
4.3	Client-Server Communication	6
5	Database Design	6
5.1	Main Relationships	7
6	UI/UX Design	7
7	User Manual and Application Flow	8
7.1	Home Page	8
7.2	Registration and Login	9
7.3	Add Product Form	10
7.4	User Dashboard	11
7.5	Product Detail Page	12
7.6	Administration Panel	12
8	Setup and Testing Instructions	13
8.1	Database Preparation	13
8.2	Starting the Backend	13
8.3	Starting the Frontend	13
8.4	Testing the Admin Role	13
8.5	Testing the User Role	14
9	Conclusions and Future Work	14
9.1	Difficulties Encountered	14
9.2	Future Work	15

1 Introduction and Objectives

The project *EcoMarket* was created with the objective of providing a modern, intuitive and secure web platform for buying and selling second-hand products. In a global context increasingly focused on sustainability and circular economy, the application aims to encourage the reuse of goods, reduce waste and promote more conscious purchasing habits.

The target users are people who want either to sell objects they no longer use or to find second-hand products at convenient prices. Special attention was paid to usability, so that even users with limited technical experience can browse the platform, publish products and manage purchase requests without difficulty.

EcoMarket addresses common problems found in local second-hand marketplaces, such as fragmented information, limited control over the transaction process and lack of trust between users. For this reason, the system integrates product management, transaction state management, differentiated user roles and a review system that improves transparency and reliability.

1.1 Technology Summary

The project was implemented as a full-stack web application using the following technologies:

- **Frontend:** React, Vite, React Router, Bootstrap 5, custom CSS and Axios.
- **Backend:** Node.js, Express and REST APIs.
- **Database:** MySQL.
- **Authentication:** JWT-based authentication mechanism.
- **Image upload:** Multer middleware for managing multipart/form-data requests.
- **Server-side rendered admin area:** Handlebars templates for the administration views.

1.2 General System Diagram

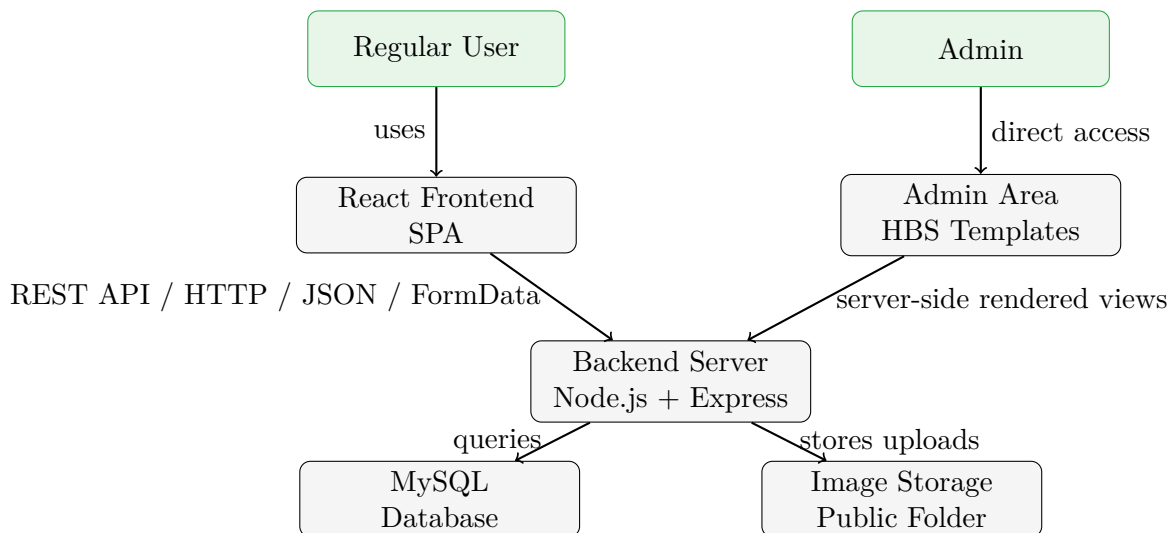


Figure 1: General architecture of the EcoMarket system, separating the React user interface from the server-side rendered administration area.

2 Work Distribution and Task Table

Since the project was developed by a team of two people, the work was divided in order to parallelize development while maintaining strong communication between the backend and frontend interfaces. The total workload was estimated at approximately 100 hours for the complete full-stack project, distributed between Sara Macrì and Danijel Merlo.

Both members worked on frontend and backend code, but with differentiated responsibilities in order to optimize time and avoid unnecessary overlap. The following table summarizes the work distribution across the ten main phases of the project.

Phase	Task	Main Assignee
Phase 1: Project Setup and Basic Server — 5 hours		
Phase 1.1	Initialize Node.js and Express.	Danijel Merlo
Phase 1.2	Define basic routing for <code>/api</code> and <code>/</code> , and set up React in the frontend.	Danijel Merlo
Phase 2: Database Configuration and Basic Seed — 10 hours		
Phase 2.1	Create the MySQL DDL schema in <code>schema.sql</code> for the main entities.	Sara Macrì
Phase 2.2	Implement the database connection script in <code>db.js</code> .	Sara Macrì
Phase 3: Backend HBS Views, Authentication and Admin Dashboard — 10 hours		
Phase 3.1	Implement JWT/cookie-based authentication support.	Sara Macrì
Phase 3.2	Develop the HBS admin panel for managing users and transactions.	Sara Macrì
Phase 4: Complete REST API Backend — 15 hours		
Phase 4.1	Implement product catalogue endpoints with filters and pagination.	Sara Macrì
Phase 4.2	Implement transaction and review endpoints.	Danijel Merlo
Phase 5: React Frontend Base Architecture, Router and Auth Context — 10 hours		
Phase 5.1	Initialize React Router DOM and define the main routes.	Danijel Merlo
Phase 5.2	Develop the Context API for authentication through <code>AuthContext</code> .	Sara Macrì
Phase 5.3	Create the API service layer in <code>api.js</code> .	Sara Macrì
Phase 5.4	Initialize the main CSS files, such as <code>index.css</code> and <code>App.css</code> .	Danijel Merlo
Phase 5.5	Test basic navigation and protected routes through <code>PrivateRoute</code> .	Sara Macrì
Phase 6: React Frontend Public Views — 15 hours		
Phase 6.1	Create the global responsive Navbar and Footer.	Sara Macrì
Phase 6.2	Develop the Home Page and catalogue using CSS Grid.	Sara Macrì
Phase 6.3	Connect filters to the API and manage pagination.	Sara Macrì
Phase 6.4	Build the product detail view.	Sara Macrì
Phase 6.5	Prepare the modal or interaction block for making a purchase request.	Sara Macrì
Phase 7: React Frontend Private Views — 15 hours		
Phase 7.1	Build the user dashboard with tab components.	Sara Macrì
Phase 7.2	Implement the Add Product form with image preview.	Sara Macrì
Phase 7.3	Display incoming purchase requests and allow the seller to accept or reject them.	Danijel Merlo

Phase	Task	Main Assignee
Phase 7.4	Allow the user to leave a review or rating after a transaction.	Danijel Merlo
Phase 8: Premium UI and Missing Pages — 10 hours		
Phase 8.1	Implement the interactive Login page.	Sara Macrì
Phase 8.2	Implement the interactive Registration page.	Danijel Merlo
Phase 8.3	Complete Footer links and static pages.	Danijel Merlo
Phase 8.4	Refine micro-interactions, such as hover effects and loading spinners.	Sara Macrì
Phase 8.5	Unify the color palette and fonts to achieve a premium visual style.	Sara Macrì
Phase 9: Final Seed, End-to-End Testing and Bug Fixing — 5 hours		
Phase 9.1	Populate the local MySQL database with realistic and dense test data.	Sara Macrì
Phase 9.2	Perform general end-to-end testing: registration, product publication, purchase flow and dashboard use.	Both members
Phase 9.3	Fix remaining JavaScript console errors or warnings.	Both members
Phase 10: Report Writing and Requirement Verification — 5 hours		
Phase 10.1	Write the project report.	Both members
Phase 10.2	Check documentation and verify project requirements.	Both members

Table 1: Work distribution and task assignment.

3 System Requirements

3.1 Functional Requirements

- **Authentication:** Login is required to perform actions on the platform, such as buying, selling and reviewing products. Although the project statement allows users to be pre-loaded in the database, we implemented and integrated a complete registration system as an additional functionality.
- **Roles and permissions:** The system distinguishes between a regular *User*, who interacts with the marketplace, and an *Admin*. The administrator has dedicated views to supervise registered users and monitor all purchase transactions performed on the platform.
- **Product management:** Authenticated users can publish a new product by entering title, description, price, condition, category, shipping options and multiple images. Product owners can edit textual information, hide their listings or permanently delete them.
- **Catalogue and search:** The platform provides a public catalogue that can also be browsed by non-authenticated users. It includes advanced filters by category, condition and price range, together with a textual search bar. Results are displayed using pagination.
- **Transactions:** A user can request the purchase of a product. The seller can accept or reject the request. Once accepted, the transaction can be completed.
- **Reviews:** After a completed transaction, the buyer can leave a review from 1 to 5 stars and a textual comment about the seller. This contributes to the seller’s public average rating.
- **Newsletter Subscription:** Users can subscribe to the platform’s newsletter. The frontend implements strict Regex validation to ensure proper email formatting and utilizes

the browser's Local Storage to prevent duplicate subscriptions from the same user.

- **Advanced Product Images Management:** When creating a new product listing, users can upload up to five images and dynamically select which one will be set as the primary "cover" image for the catalogue.
- **Optimized Search Functionality:** The textual search bar has been fine-tuned to filter products exclusively by their title rather than their description, ensuring highly relevant and precise search results for the users.
- **Enhanced Navigation Flow:** To improve UX, logical redirect buttons (such as returning to the dashboard after reviewing a published listing) have been integrated into the product detail views.

3.2 Non-Functional Requirements

- **Security:** Passwords are stored in the database as hashes using bcrypt. Sessions are managed through JSON Web Tokens, which are sent in HTTP headers for private requests.
- **Performance:** Database pagination is used to avoid excessive data loading. Images are loaded asynchronously, and the Single Page Application structure allows smooth transitions between pages without full reloads.
- **Usability and accessibility:** The interface is responsive and adapts to desktop, tablet and mobile screens. The application also provides clear visual feedback through loading spinners and success/error messages.

4 Architecture and Technological Choices

The system follows a client-server architecture based on REST APIs. The frontend and backend are fully decoupled and communicate through HTTP requests. JSON is used for most API calls, while FormData is used for file uploads.

4.1 Frontend

- **React with Vite:** Chosen to develop a fast and reactive Single Page Application, based on reusable components and efficient DOM updates.
- **React Router:** Used for client-side navigation without refreshing the page.
- **Context API:** Used to manage global authentication state, including the logged-in user and token.
- **Bootstrap 5 and custom CSS:** Used to ensure a responsive layout and a robust design system, refined with custom styles to give the application a premium appearance.
- **Axios:** Used to perform asynchronous calls to the backend REST APIs, with interceptors to attach the JWT token to private requests.

4.2 Backend

- **Node.js with Express:** Used to create a lightweight, asynchronous and scalable server.
- **MVC-inspired structure:** The backend follows a logical structure similar to Model-View-Controller, adapted to REST APIs:
 - `routes/`: defines endpoints and maps requests.
 - `controllers/`: contains the business logic.

- **middleware/**: contains cross-cutting functionality, such as authentication and file upload.
- **Multer**: Middleware used to manage image uploads through **multipart/form-data**, storing files locally inside the **public** folder.
- **Handlebars**: Used for the server-side rendered administration views.

4.3 Client-Server Communication

The communication between the React frontend and the Node.js backend is entirely based on the HTTP protocol over a RESTful architecture.

- **Data Exchange**: Standard requests (such as fetching the catalogue or registering a user) transmit data formatted in **JSON**. The frontend utilizes the **Axios** library to perform these asynchronous calls.
- **File Uploads**: For product creation, which includes image uploads, the frontend switches to **FormData** objects. The backend parses these specific requests using the **Multer** middleware.
- **Security & State**: Since the REST API is stateless, authentication is maintained via **JWT (JSON Web Tokens)**. The frontend attaches the token to the HTTP Authorization header of every private request, ensuring secure communication between the two distinct environments.

5 Database Design

The relational database used in the project is MySQL. The schema contains the following main entities:

- **Users**: id, name, surname, email, password_hash, role, created_at.
- **Categories**: id, name, description, icon.
- **Products**: id, title, description, price, condition, status, views_count, shipping_available, pickup_location, seller_id, category_id.
- **Product_Images**: id, product_id, image_url, is_primary.
- **Transactions**: id, product_id, buyer_id, seller_id, agreed_price, status.
- **Reviews**: id, transaction_id, reviewer_id, reviewee_id, rating, comment.

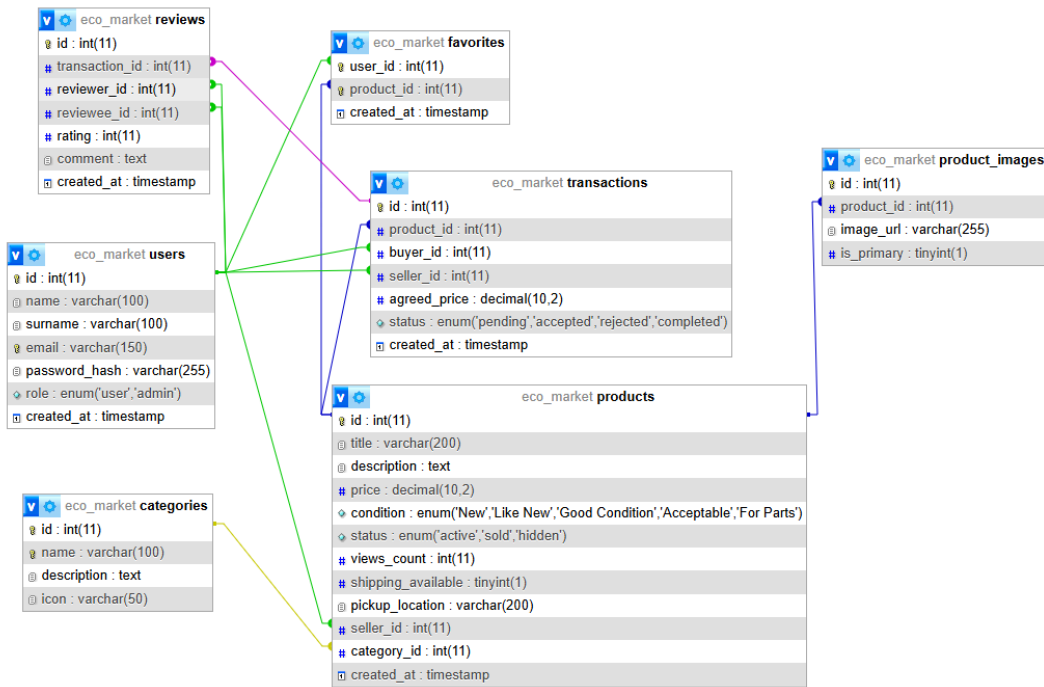


Figure 2: Entity-Relationship diagram illustrating the MySQL database scheme.

5.1 Main Relationships

- **Users and Products:** One user, acting as seller, can publish many products. Each product belongs to one seller.
- **Products and Product Images:** One product can have from one to five images. If a product is deleted, its image records are deleted through `ON DELETE CASCADE`.
- **Transactions and Reviews:** Each transaction connects one buyer, one seller and one product. A completed transaction can generate at most one review per user.

6 UI/UX Design

The user interface was designed to be elegant, clean and professional. The goal was to create a premium-looking marketplace while keeping the user experience simple and intuitive.

- **Color palette and typography:** A green primary color was chosen to reinforce the ecological identity of the application. It is balanced with white space and light gray tones to highlight the products and avoid visual overload.
- **Hover effects:** Buttons and product cards slightly rise through `box-shadow` and `translateY` effects when the user moves the mouse over them, creating a more interactive feeling.
- **Dynamic images:** When a product card contains more than one image, the interface can display them dynamically, improving the visual presentation of the catalogue.
- **Loading states:** Animated spinners are displayed while data is loading, preventing the user from thinking that the application is blocked.

7 User Manual and Application Flow

This section describes the user journey through the main screens of the application.

7.1 Home Page

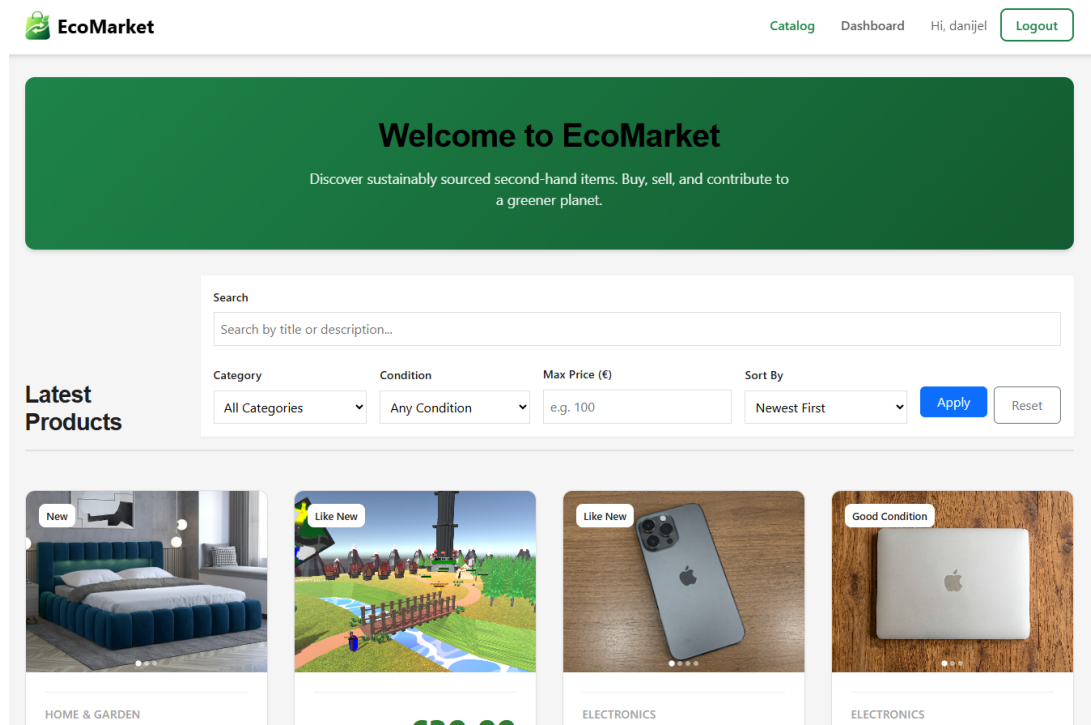
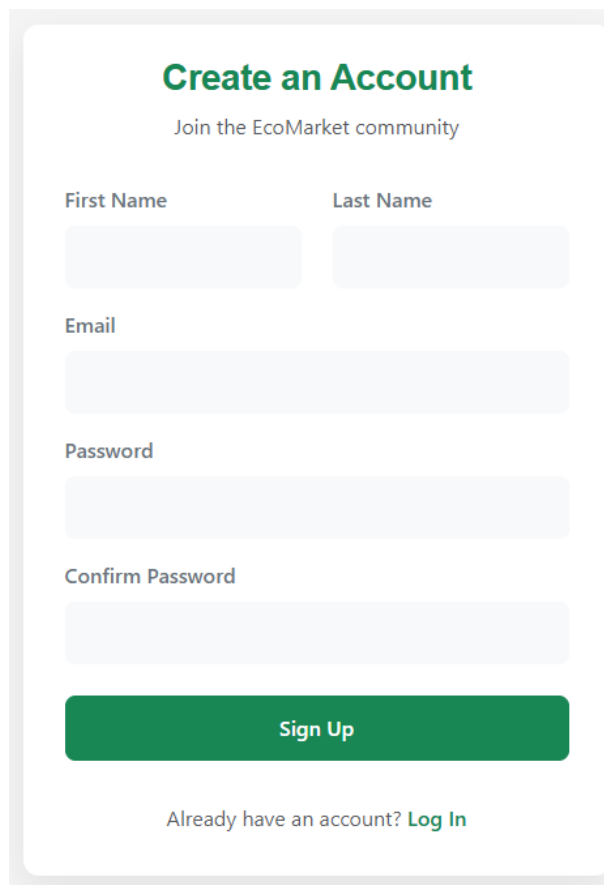


Figure 3: EcoMarket Home Page and catalog view with search filters.

The user arrives at the Home Page and can freely explore the catalogue. Products can be found through the search bar or through the available filters, such as category and price.

7.2 Registration and Login



Create an Account
Join the EcoMarket community

First Name

Last Name

Email

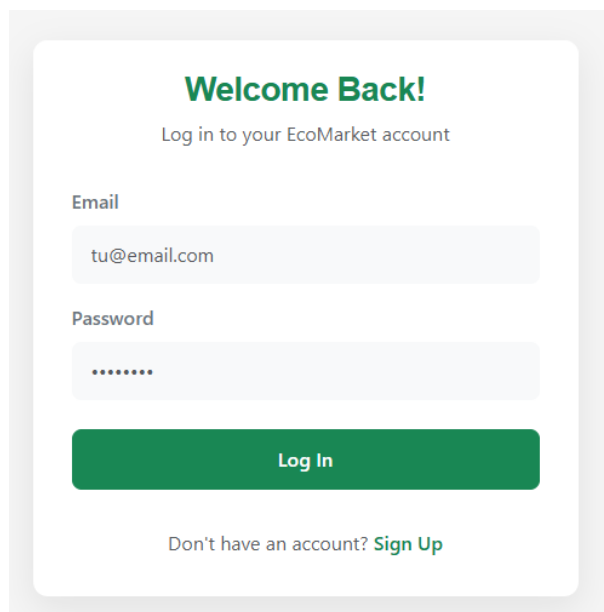
Password

Confirm Password

Sign Up

Already have an account? [Log In](#)

Figure 4: The registration interface, allowing new users to create an account through a form with custom frontend validation.



Welcome Back!
Log in to your EcoMarket account

Email

Password

Log In

Don't have an account? [Sign Up](#)

Figure 5: The secure login page required for users to authenticate and gain access to the marketplace features.

7.3 Add Product Form

From the dashboard, the user can click on *Add Product*. A form is displayed where they can insert the product details, select the condition from a dropdown menu, and upload up to five photos with real-time preview.

Add a New Product

Listing Title *

King Size Bed

Description *

New perfect king size bed

Price (€) *

890

Condition *

New

Category *

Home & Garden

☐ Shipping Available

Pickup Location (Optional)

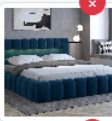
Milano, Via Roma

Product Images *

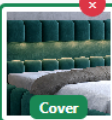
Scegli file | Nessun f...lezzionato

You can upload up to 5 images. Select one to set as cover.

Preview (Select Cover):







Cancel

Publish Listing

Figure 6: The product creation form, featuring input fields for item specifications.

7.4 User Dashboard

In the dashboard, the users have full control over their activity. They can monitor sales, accept or reject purchase requests, and edit or delete their own listings.

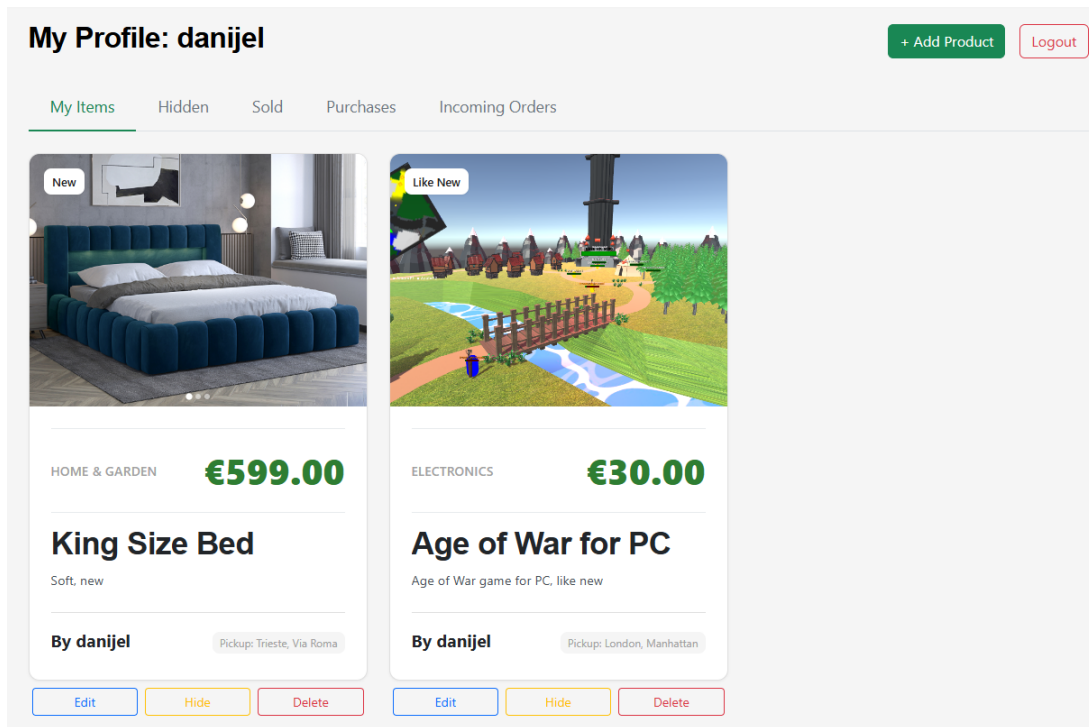


Figure 7: The private user dashboard, providing a centralized control panel.

7.5 Product Detail Page

When a product is selected, the detail page is opened. From this page, the buyer can contact the seller or directly start the purchase request. The page clearly displays the image gallery, seller reliability through stars, and shipping availability.

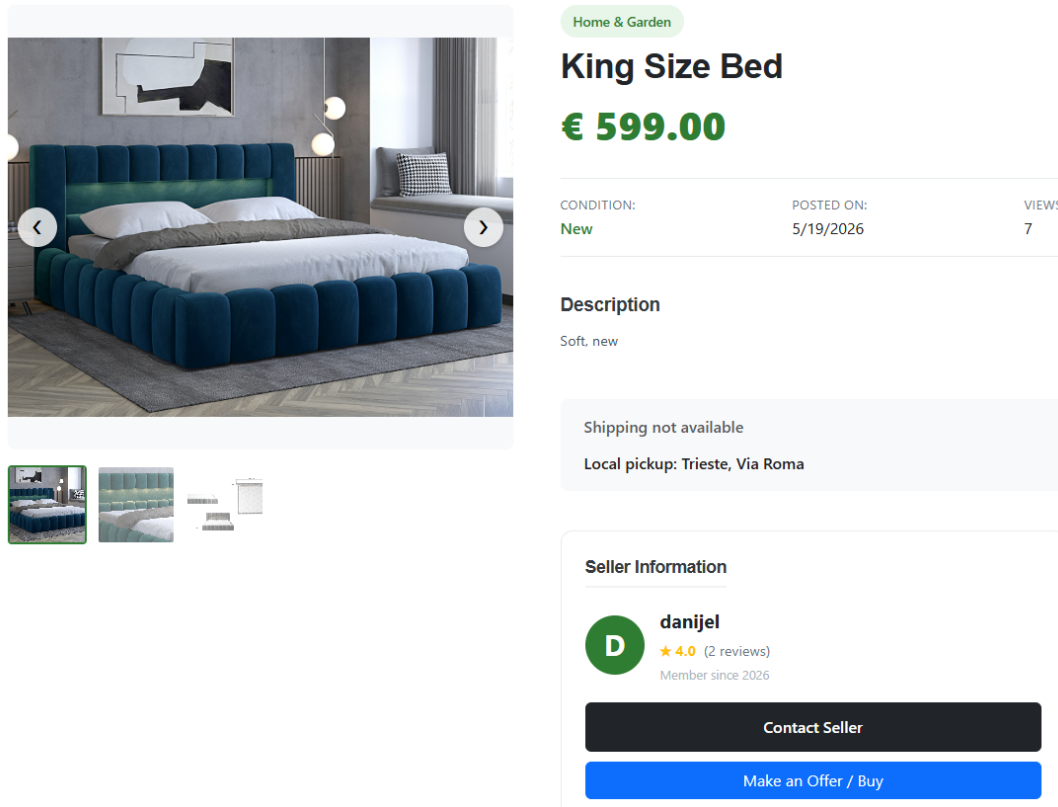


Figure 8: The detailed product view from a buyer's perspective.

7.6 Administration Panel

Users with the *Admin* role have access to a separate administration dashboard. From this area, they can view the complete list of registered users and monitor the status of all purchase transactions, ensuring centralized control over the activity of the platform.

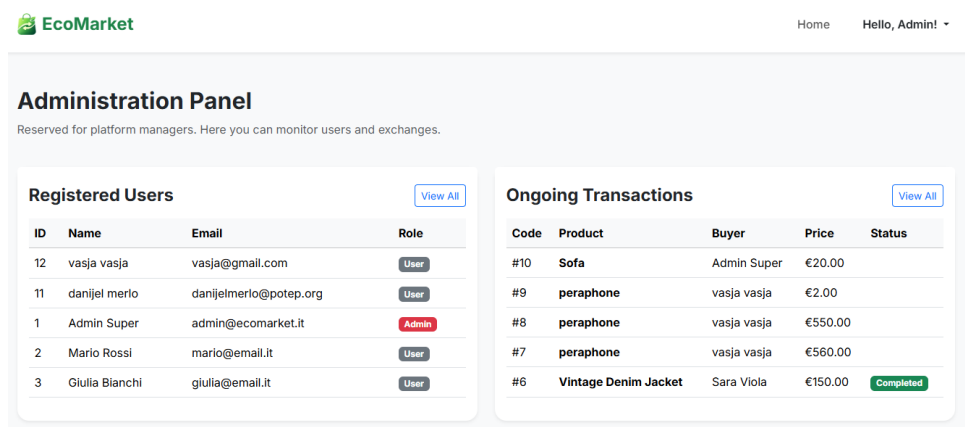


Figure 9: The server-side rendered administration panel.

8 Setup and Testing Instructions

Since EcoMarket is a full-stack project, the backend and frontend must be started separately.

8.1 Database Preparation

The following steps are required only for the first time. MySQL must be running before importing the database schema and seed data.

```
brew services start mysql
cd backend
mysql -u root -e "CREATE DATABASE IF NOT EXISTS eco_market;"
mysql -u root eco_market < schema.sql
node seed.js
```

8.2 Starting the Backend

Open a first terminal, enter the backend folder, install the dependencies if necessary, and start the server.

```
cd backend
npm install
node server.js
```

The backend will be available at:

`http://localhost:3000`

This terminal must remain open while testing the application.

8.3 Starting the Frontend

Open a second terminal, enter the frontend folder, install the dependencies if necessary, and start Vite.

```
cd frontend
npm install
npm run dev
```

The frontend will be available at:

`http://localhost:5173`

This terminal must also remain open while testing the application.

8.4 Testing the Admin Role

As required by the evaluation criteria, the administration area was implemented using server-side rendering with the Handlebars template engine. To test it:

- Do not use the React interface.
- Navigate directly to the backend server:

`http://localhost:3000/login`

Then log in using the following pre-loaded credentials:

Email: `admin@ecomarket.it`
Password: `password123`

8.5 Testing the User Role

To test the platform as a regular user:

- Navigate to the frontend interface:

`http://localhost:5173`

A user can log in with one of the accounts generated by the seed file, for example:

Email: `mario@email.it`

Password: `password123`

Alternatively, a new account can be created through the additional registration functionality.

9 Conclusions and Future Work

EcoMarket achieved the main objectives of the project by providing a working platform for the core features of a second-hand marketplace, including product CRUD operations, authentication, transaction management, and reviews.

9.1 Difficulties Encountered

One of the main challenges was managing multiple file uploads. This required coordination between the frontend, which sends data using `FormData`, and the backend, where Multer processes the uploaded files, stores them in a folder, and saves their paths in the database.

Another important difficulty was the secure management of sessions. This was solved by applying JWT tokens and implementing Axios interceptors to automatically attach the token to private requests.

Project Size Optimization and Academic Evaluation Adjustments During the final packaging of the application, we encountered a submission constraint restricting the project archive (ZIP file) to a maximum of 100 MB. Because the database was seeded with a high volume of products and image assets, the initial project folder exceeded 200 MB. Since the system is designed to preserve images of sold products so that buyers can continue to view them in their personal purchased history catalog, removing these images permanently was not an option.

To resolve this issue and meet the size limit:

- **Placeholder Image Fallback:** We removed redundant large images from the assets and implemented a fallback mechanism. When a product has no associated images, the system displays a lightweight placeholder image (`placeholder.png`, located at `frontend/public/placeholder.png`). This logic is implemented in `frontend/src/components/ProductCard.jsx` (lines 31–39) and `frontend/src/pages/ProductDetail.jsx` (lines 52–55):

```
1 const getImageUrl = (url) => {  
2   if (!url) return '/placeholder.png';  
3   return url.startsWith('http') ? url : `http://localhost:3000${url}`;  
4 };
```

- **Demonstration of Database Density:** For evaluation purposes, we kept sold items visible in the public catalog, marked with a `SOLD` badge. This allows the evaluator to appreciate the full database state. In `frontend/src/pages/Home.jsx` (line 29), this is controlled by passing `status: 'all'` in the request parameters:

```
1 const params = {  
2   page: page,  
3   limit: 12,  
4   status: 'all' // Displays sold items for demonstration  
5 };
```

In a real production environment, removing sold items from the public homepage catalog is easily achieved by commenting out or removing the `status: 'all'` parameter (or setting it to `'available'`).

9.2 Future Work

Several improvements could be implemented in future versions of the project:

- **Payment Gateway Integration:** For example, Stripe could be integrated to manage payments directly through the platform. This would allow the system to securely process transactions, hold funds in escrow, and automate payments once the item is delivered.
- **Purchase and Logistics Management:** The application could be expanded to include a comprehensive checkout flow. This would allow buyers and sellers to officially formalize the purchase by selecting and tracking the fulfillment method, choosing between courier shipping or local hand-delivery.
- **Counter-Offer Mechanism:** The current offer system could be upgraded to support dynamic negotiations. Buyers could propose a different price, and sellers would have the ability to accept, reject, or send a counter-offer before finalizing the deal.
- **Newsletter Management System:** While the current platform validates and registers email subscriptions, a dedicated backend module is needed to actively manage marketing campaigns, send scheduled newsletters, and handle unsubscriptions.
- **Detailed Review Display:** The system currently processes reviews to calculate and display a seller's average rating. A future implementation should include a dedicated user interface on the seller's public profile to display the individual textual reviews and specific ratings left by past buyers.
- **Real-time Chat System:** WebSockets or Socket.io could be used to allow buyers and sellers to arrange pickup details directly within the platform.
- **Password Recovery:** A recovery system could be implemented through email using Nodemailer.
- **Notifications:** Users could receive push or email notifications when a new purchase request is received or when the status of an order changes.

Overall, EcoMarket represents a complete full-stack web application that applies the main technologies studied in the course and provides a solid basis for future extensions.